Lembar Kerja (Post-Test) Modul 11

Soal Nomor 1 Identifikasi dan perbandingan minimal tiga DBMS

DBMS (Database Management System) adalah perangkat lunak yang digunakan untuk membuat, mengelola, dan mengakses basis data. Jawaban ini membandingkan tiga DBMS yang mewakili model data berbeda, yaitu MySQL/MariaDB (relasional), PostgreSQL (objek-relasional), dan MongoDB (berorientasi dokumen).

1.1 Hasil Identifikasi DBMS

Aspek	MySQL/ MariaDB	PostgreSQL	MongoDB
Model data	Relasional	Objek-relasional	Berorientasi dokumen (NoSQL)
Bentuk penyimpanan	Tabel (baris dan kolom)	Tabel, dengan tipe kompleks (array, JSON/JSONB, tipe buatan sendiri)	Dokumen BSON/JSON dalam collection
Bahasa kueri	SQL	SQL (cakupan standar lebih lengkap)	MongoDB Query Language (MQL) dan aggregation pipeline
Skema	Tetap	Tetap, fleksibel lewat JSONB	Fleksibel (schema-less)
Lisensi	Open source (MySQL: GPL, dikelola Oracle; MariaDB:	Open source (lisensi PostgreSQL)	Open source (SSPL) dan layanan cloud (Atlas)

Aspek	MySQL/ MariaDB	PostgreSQL	MongoDB
	GPL, komunitas)		
Dukungan JSON	MySQL: JSON native; MariaDB: berbasis teks dengan validasi	JSON dan JSONB (dapat diindeks)	Format utama (BSON)
Skalabilitas	Vertikal, replikasi master-replica	Vertikal, replikasi, partitioning	Horizontal lewat sharding dan replica set
Alat bantu	phpMyAdmin, MySQL Workbench	pgAdmin	MongoDB Compass

1.2 Karakteristik, Kelengkapan Fitur, Kelebihan, dan Keterbatasan

a. MySQL / MariaDB

DBMS relasional yang ringan dan populer, banyak dipakai pada aplikasi web. Data disimpan dalam tabel yang dihubungkan dengan primary key dan foreign key.

Kelengkapan fitur: SQL standar, transaksi ACID (mesin penyimpanan InnoDB), foreign key dan constraint, stored procedure, trigger, view, indeks, manajemen pengguna dan hak akses, backup dan restore, replikasi, serta tipe data JSON.

Kelebihan:

- Mudah dipasang, dipelajari, dan digunakan (tersedia phpMyAdmin).
- Gratis, dengan komunitas besar dan dokumentasi lengkap.
- Kinerja baik untuk aplikasi web dengan data terstruktur.
- Didukung hampir semua layanan hosting dan bahasa pemrograman.

Keterbatasan:

- Fitur tingkat lanjut (tipe data kompleks, analitik rumit) tidak selengkap PostgreSQL.
- Skema kaku sehingga kurang cocok untuk data yang strukturnya sering berubah.
- Skalabilitas horizontal tidak semudah basis data NoSQL.
- Implementasi JSON pada MySQL dan MariaDB tidak sepenuhnya identik.

b. PostgreSQL

DBMS objek-relasional yang kuat, patuh pada standar SQL, dan dapat diperluas (extensible). Selain data relasional, PostgreSQL mendukung konsep objek seperti pewarisan tabel dan tipe data buatan sendiri.

Kelengkapan fitur: ACID dengan MVCC (Multi-Version Concurrency Control), JSONB dengan indeks, array, window function, common table expression, full-text search, stored procedure, trigger, replikasi, partitioning, serta ekstensi seperti PostGIS untuk data geografis.

Kelebihan:

- Fitur sangat lengkap dan mampu menangani kueri kompleks serta analitik.
- Mendukung data terstruktur dan semi-terstruktur sekaligus.
- Integritas dan keandalan data tinggi.
- Gratis, open source, dan dapat diperluas dengan ekstensi.

Keterbatasan:

- Konfigurasi dan pemeliharaan lebih kompleks dibanding MySQL.
- Kurva belajar lebih curam bagi pemula.
- Membutuhkan penyetelan (tuning) agar kinerja optimal pada beban tertentu.

c. MongoDB

DBMS NoSQL berorientasi dokumen. Data disimpan sebagai dokumen BSON yang strukturnya boleh berbeda antar dokumen, sehingga cocok untuk data semi-terstruktur.

Kelengkapan fitur: skema fleksibel, indeks, aggregation pipeline, replikasi (replica set), sharding, pencarian teks, transaksi multi-dokumen pada versi yang lebih baru, serta layanan cloud (Atlas).

Kelebihan:

- Skema fleksibel sehingga perubahan struktur data mudah dilakukan.
- Skalabilitas horizontal baik untuk data berskala sangat besar.
- Cocok untuk aplikasi yang cepat berkembang dan data berformat JSON.
- Format dokumen mirip objek pada kode program sehingga mudah dipakai pengembang.

Keterbatasan:

- Relasi antardata tidak sekuat basis data relasional dan penggabungan data (join) kurang praktis.
- Rentan terhadap duplikasi data karena data sering ditanam (embedded).
- Kurang ideal untuk transaksi kompleks yang sangat bergantung pada konsistensi banyak entitas.
- Menggunakan bahasa kueri yang berbeda dari SQL.

1.3 Perbandingan

Kriteria	MySQL / MariaDB	PostgreSQL	MongoDB
Kemudahan penggunaan	Sangat mudah	Sedang	Mudah (bagi pengembang)
Kelengkapan fitur SQL	Baik	Sangat lengkap	Tidak menggunakan SQL

Kriteria	MySQL / MariaDB	PostgreSQL	MongoDB
Konsistensi dan integritas data	Kuat	Sangat kuat	Cukup (fokus pada fleksibilitas)
Fleksibilitas struktur data	Rendah (ada JSON)	Sedang sampai tinggi (JSONB)	Sangat tinggi
Skalabilitas horizontal	Terbatas	Sedang	Sangat baik
Cocok untuk	Aplikasi web dan sistem informasi	Sistem kompleks, analitik, data geografis	Data besar dan semi-terstruktur

1.4 Alasan Pemilihan DBMS yang Paling Sesuai

Organisasi yang dijadikan contoh adalah pengelola sistem Green Map PNM. Kebutuhannya: data tanaman, kategori, perawatan, dan laporan kondisi bersifat terstruktur dan saling berhubungan; data harus konsisten dan aman; anggaran terbatas; serta pengelola dan mahasiswa sudah mengenal SQL.

DBMS yang dipilih: MySQL/MariaDB. Alasannya:

1. **Sesuai sifat data.** Data berbentuk tabel yang saling berhubungan, sehingga model relasional dengan primary key dan foreign key menjaga konsistensi data.
2. **Integritas dan keamanan terjamin.** Tersedia transaksi ACID, constraint, serta pengaturan hak akses.
3. **Biaya rendah.** Bersifat open source sehingga tidak membutuhkan biaya lisensi.
4. **Mudah dipelajari dan dikelola.** Tersedia phpMyAdmin dan dokumentasi yang lengkap.
5. **Cukup fleksibel.** Tipe data JSON memungkinkan penyimpanan atribut yang bervariasi tanpa harus berpindah ke basis data NoSQL.

PostgreSQL lebih tepat jika organisasi membutuhkan analitik kompleks atau data geografis, sedangkan MongoDB lebih sesuai untuk data berskala sangat besar dengan struktur yang sering berubah.

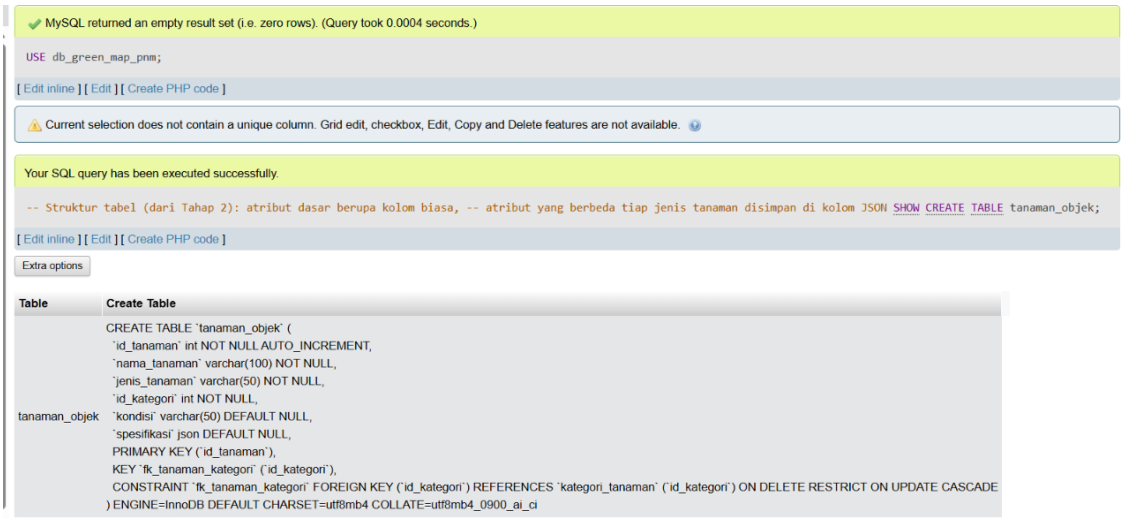
Soal Nomor 2 Implementasi penyimpanan JSON (INSERT, SELECT, UPDATE, DELETE)

Praktik dilakukan pada tabel tanaman_objek di database db_green_map_pnm. Atribut dasar tanaman disimpan pada kolom biasa, sedangkan atribut yang berbeda tiap jenis tanaman disimpan pada kolom JSON bernama spesifikasi.

2.1 Struktur Tabel

```
SHOW CREATE TABLE tanaman_objek;
```

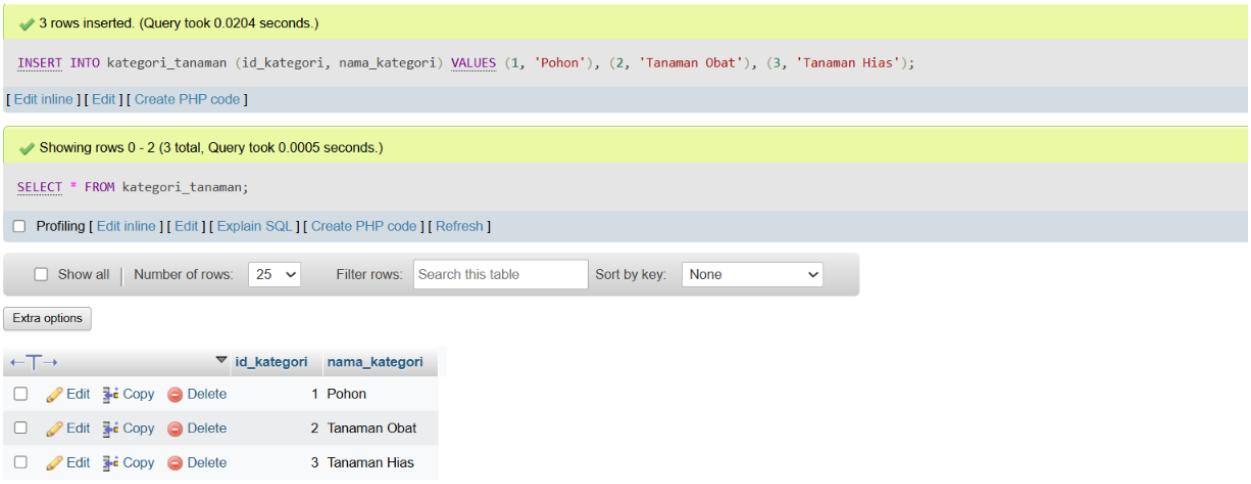
```
CREATE TABLE `tanaman_objek` (
  `id_tanaman` int NOT NULL AUTO_INCREMENT,
  `nama_tanaman` varchar(100) NOT NULL,
  `jenis_tanaman` varchar(50) NOT NULL,
  `id_kategori` int NOT NULL,
  `kondisi` varchar(50) DEFAULT NULL,
  `spesifikasi` json DEFAULT NULL,
  PRIMARY KEY (`id_tanaman`),
  KEY `fk_tanaman_kategori` (`id_kategori`),
  CONSTRAINT `fk_tanaman_kategori` FOREIGN KEY (`id_kategori`)
    REFERENCES `kategori_tanaman` (`id_kategori`)
    ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;
```



2.2 Persiapan: Mengisi Tabel Kategori

Percobaan awal memasukkan data tanaman menghasilkan error #1452 karena tabel kategori_tanaman masih kosong dan foreign key tidak menemukan kategori yang dirujuk. Masalah diselesaikan dengan mengisi kategori terlebih dahulu.

```
INSERT INTO kategori_tanaman (id_kategori, nama_kategori) VALUES
(1, 'Pohon'),
(2, 'Tanaman Obat'),
(3, 'Tanaman Hias');
SELECT * FROM kategori_tanaman;
```



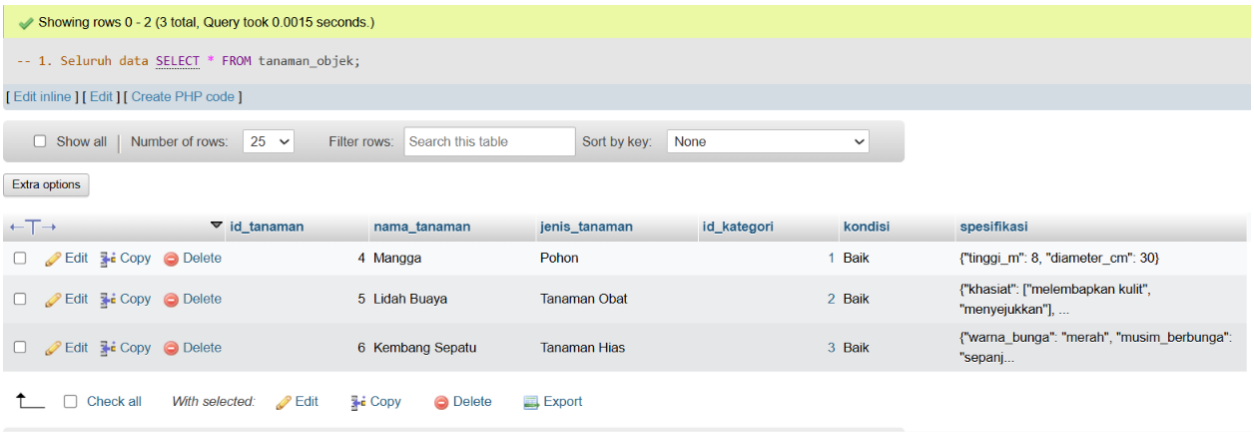
2.3 INSERT (Menambah Data)

```
INSERT INTO tanaman_objek (nama_tanaman, jenis_tanaman, id_kategori, kondisi,
spesifikasi)
VALUES
('Mangga', 'Pohon', 1, 'Baik',
    JSON_OBJECT('tinggi_m', 8, 'diameter_cm', 30)),
('Lidah Buaya', 'Tanaman Obat', 2, 'Baik',
    JSON_OBJECT('bagian_dipakai', 'daun',
                'khasiat', JSON_ARRAY('melembapkan kulit', 'menyejukkan'))),
('Kembang Sepatu', 'Tanaman Hias', 3, 'Baik',
    JSON_OBJECT('warna_bunga', 'merah', 'musim_berbunga', 'sepanjang tahun'));
```



2.4 SELECT (Membaca Data)

```
SELECT * FROM tanaman_objek;
```



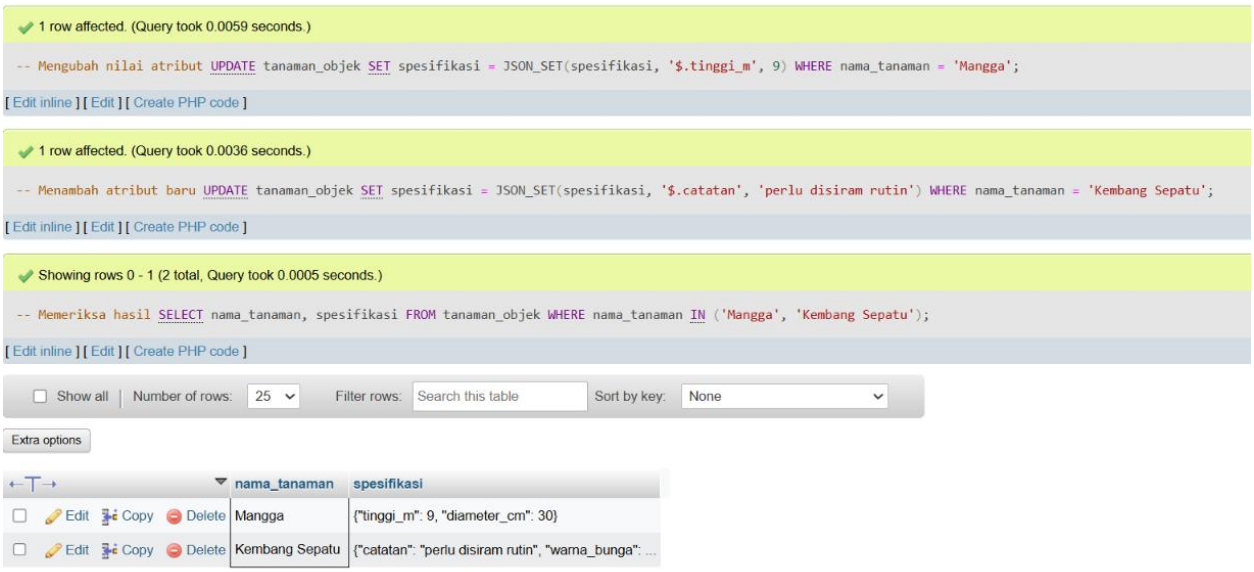
2.5 UPDATE (Mengubah Data JSON)

```
-- Mengubah nilai atribut yang sudah ada
UPDATE tanaman_objek
```

```
SET spesifikasi = JSON_SET(spesifikasi, '$.tinggi_m', 9)
WHERE nama_tanaman = 'Mangga';

-- Menambah atribut baru
UPDATE tanaman_objek
SET spesifikasi = JSON_SET(spesifikasi, '$.catatan', 'perlu disiram rutin')
WHERE nama_tanaman = 'Kembang Sepatu';

-- Memeriksa hasil
SELECT nama_tanaman, spesifikasi FROM tanaman_objek
WHERE nama_tanaman IN ('Mangga', 'Kembang Sepatu');
```



2.6 DELETE (Menghapus Data)

```
-- Menghapus satu atribut di dalam JSON
UPDATE tanaman_objek
SET spesifikasi = JSON_REMOVE(spesifikasi, '$.diameter_cm')
WHERE nama_tanaman = 'Mangga';

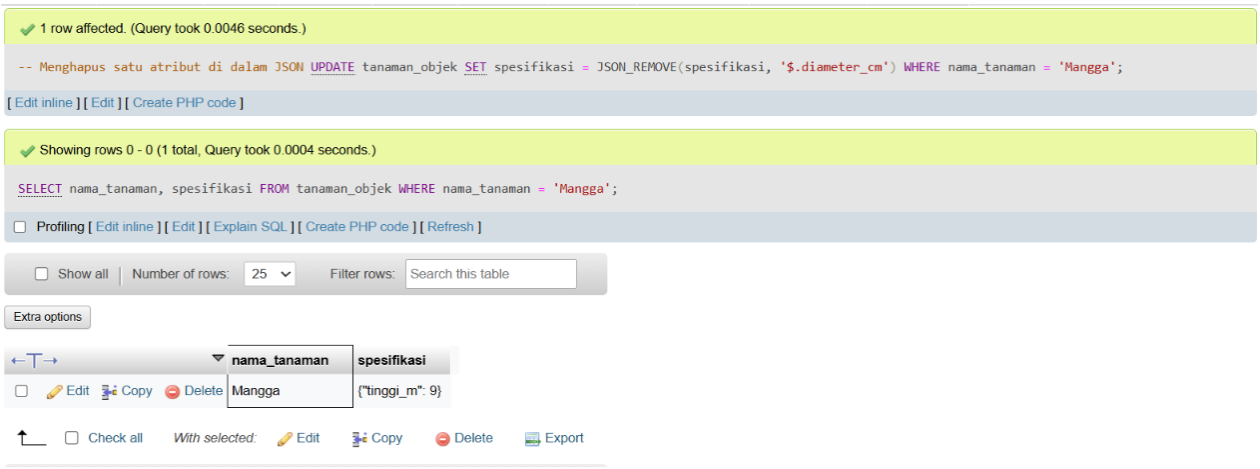
SELECT nama_tanaman, spesifikasi FROM tanaman_objek
WHERE nama_tanaman = 'Mangga';

-- Menghapus satu baris data
```

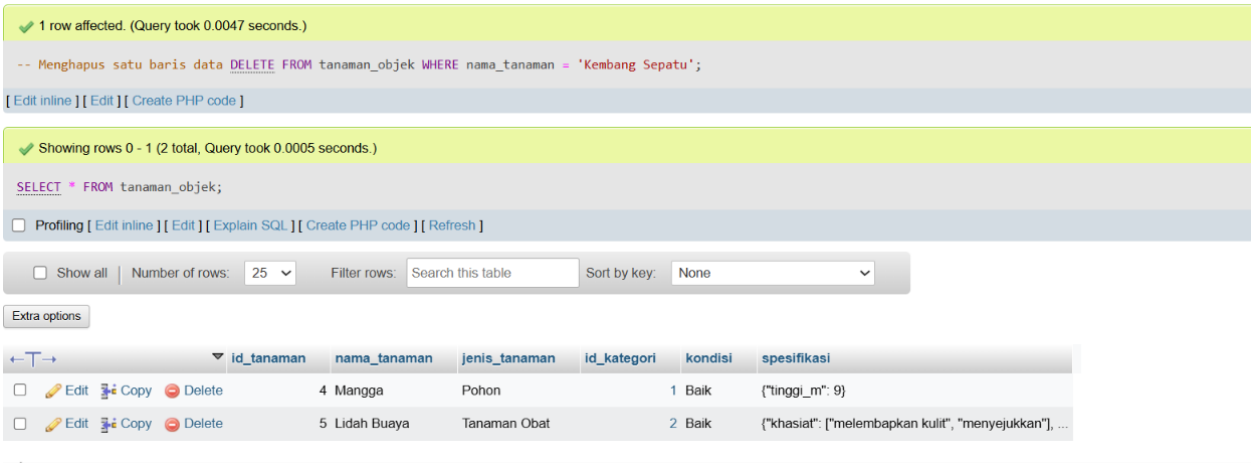
```
DELETE FROM tanaman_objek WHERE nama_tanaman = 'Kembang Sepatu';


SELECT * FROM tanaman_objek;
```

Hasil JSON_REMOVE pada atribut diameter_cm



Hasil DELETE baris Kembang Sepatu dan SELECT akhir



2.7 Manfaat JSON untuk Data dengan Struktur Bervariasi

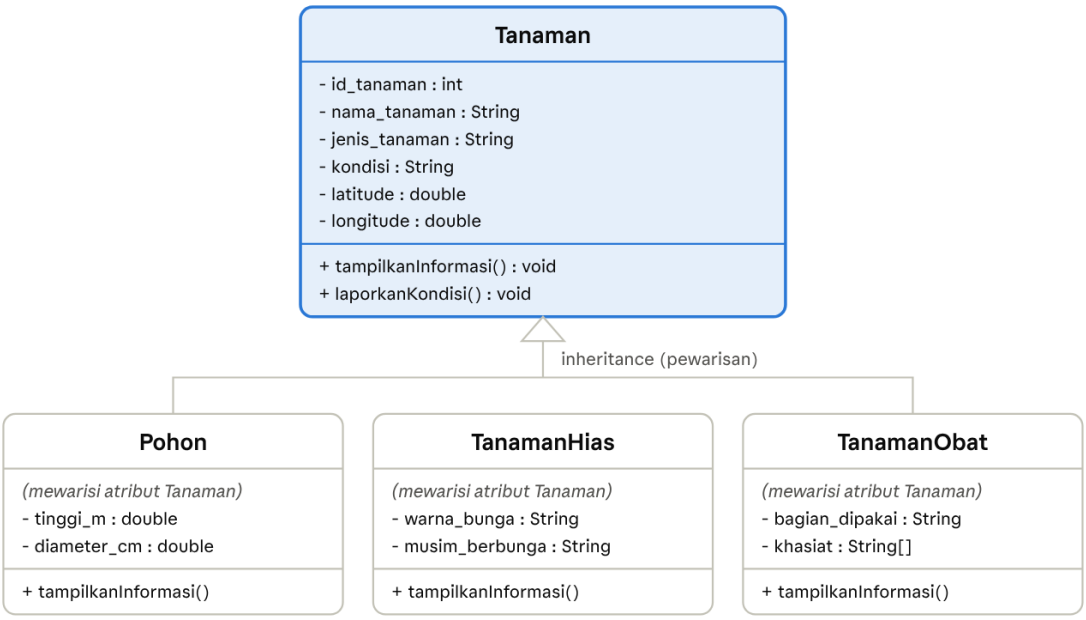
1. **Menampung atribut yang berbeda tiap jenis tanaman.** Pohon memerlukan tinggi dan diameter, tanaman hias memerlukan warna bunga, dan tanaman obat memerlukan bagian yang dipakai serta khasiat. Semuanya cukup disimpan dalam satu kolom JSON.
2. **Menghindari banyak kolom kosong (NULL).** Tanpa JSON, tabel harus memiliki seluruh kolom atribut untuk semua jenis tanaman, padahal sebagian besar kosong pada setiap baris.
3. **Mudah menambah atau mengubah atribut.** Atribut baru ditambahkan dengan JSON_SET tanpa ALTER TABLE, sehingga struktur tabel dan aplikasi tidak terganggu.
4. **Mendukung data bersarang dan array.** Daftar khasiat dapat disimpan sebagai satu array tanpa tabel tambahan.
5. **Tetap dapat dicari dan diolah dengan SQL.** Fungsi seperti JSON_EXTRACT, JSON_CONTAINS, dan JSON_LENGTH memungkinkan pencarian di dalam JSON dan dapat digabung dengan kolom biasa.
6. **Mudah dikembangkan.** Jika muncul jenis tanaman baru dengan atribut berbeda, data langsung dapat disimpan tanpa mengubah rancangan tabel.

Soal Nomor 3 Pemodelan objek dan representasi objek pada DBMS relasional

Pemodelan objek dibuat berdasarkan objek Tanaman pada sistem Green Map PNM. Class Tanaman menjadi induk (superclass) dengan tiga class turunan sesuai jenis tanaman, yaitu Pohon, TanamanHias, dan TanamanObat.

3.1 Diagram Pemodelan Objek

Pohon, TanamanHias, dan TanamanObat mewarisi class Tanaman



Method tampilkanInformasi() di-override pada tiap subclass (polymorphism). Tanda - = private, + = public.

Penjelasan diagram:

- **Class:** Tanaman sebagai superclass; Pohon, TanamanHias, dan TanamanObat sebagai subclass.
- **Attribute:** Tanaman memiliki id_tanaman, nama_tanaman, jenis_tanaman, kondisi, latitude, dan longitude. Setiap subclass menambah atribut khusus: tinggi dan diameter (Pohon), warna dan musim berbunga (TanamanHias), serta bagian yang dipakai dan khasiat (TanamanObat).
- **Method:** tampilkanInformasi() dan laporkanKondisi() pada Tanaman.
- **Inheritance:** segitiga kosong menunjukkan bahwa subclass mewarisi atribut dan method Tanaman.

- **Polymorphism:** method tampilkanInformasi() di-override pada tiap subclass sehingga setiap jenis tanaman dapat menampilkan informasi yang berbeda.

Diagram ini adalah model objek konseptual, sehingga atributnya tidak harus identik dengan kolom pada tabel praktik.

3.2 Perbedaan Basis Data Relasional dan Berorientasi Objek

Aspek	Relasional	Berorientasi Objek	Contoh pada Green Map PNM
Struktur data	Tabel (baris dan kolom)	Objek	Tabel tanaman_objek dan objek Tanaman
Identitas	Primary key	Object identity	id_tanaman
Hubungan	Foreign key	Object reference	id_kategori merujuk ke kategori_tanaman
Atribut	Kolom	Properti objek	nama_tanaman, kondisi
Perilaku	Umumnya di aplikasi atau stored program	Menjadi bagian dari model objek	laporkanKondisi()
Pewarisan	Bukan fitur utama model relasional	Mendukung inheritance	Pohon dan TanamanHias turunan Tanaman
Contoh DBMS	MySQL, PostgreSQL	ObjectDB, db4o (historis)	Green Map PNM memakai MySQL

Secara singkat, basis data relasional memandang data sebagai tabel yang dihubungkan melalui kunci, sedangkan basis data berorientasi objek memandang data sebagai objek utuh yang membawa identitas, atribut, perilaku, dan hubungan pewarisan.

3.3 Representasi Objek pada DBMS Relasional

MySQL bukan DBMS berorientasi objek murni, sehingga objek dipetakan ke struktur relasional:

Konsep objek	Representasi di MySQL
Class	Tabel (tanaman_objek)
Attribute	Kolom (nama_tanaman, kondisi)
Object (instance)	Baris data
Object identity	Primary key (id_tanaman)
Object reference	Foreign key (id_kategori)
Method	Stored function/procedure atau kode aplikasi
Inheritance	Salah satu dari tiga strategi di bawah

Strategi memetakan inheritance:

1. **Satu tabel dengan kolom JSON.** Atribut dasar disimpan di kolom biasa dan atribut khusus tiap jenis disimpan di kolom JSON. Sederhana dan fleksibel, tetapi validasi atribut dalam JSON lebih lemah.
2. **Tabel per subclass.** Setiap subclass memiliki tabel sendiri (misalnya pohon) dengan primary key yang juga menjadi foreign key ke tabel tanaman. Struktur jelas, tetapi membutuhkan JOIN.
3. **Tabel gabungan tanpa induk.** Setiap tabel memuat semua atribut (induk dan turunan). Tidak perlu JOIN, tetapi atribut induk terduplikasi.

Pendekatan yang dipakai pada praktik: strategi pertama. Atribut dasar tanaman disimpan di kolom biasa, dan atribut khusus jenis tanaman disimpan di kolom JSON spesifikasi. Pendekatan ini tetap bersifat semi-terstruktur dan tidak mengubah MySQL menjadi basis data berorientasi objek.

Soal Nomor 4 Penerapan basis data: analisis kebutuhan, model data, fitur DBMS, dan teknologi

Bidang penerapan yang dipilih adalah pengelolaan dan pemetaan ruang hijau kampus (bidang lingkungan) melalui sistem Green Map PNM, sesuai proyek basis data kelompok.

4.1 Analisis Kebutuhan Organisasi

Pengelola ruang hijau kampus membutuhkan sistem untuk:

- mencatat data tanaman (nama, jenis, kategori, kondisi, dan lokasi);
- mengelompokkan tanaman berdasarkan kategori (pohon, tanaman obat, tanaman hias);
- mencatat riwayat perawatan tanaman;
- menerima dan menyimpan laporan kondisi tanaman;
- mengelola data pengguna dan hak aksesnya;
- menjaga data tetap konsisten, aman, dan dapat dipulihkan jika terjadi kerusakan.

Kebutuhan non-fungsional: data konsisten, akses cepat, biaya rendah, mudah dikelola oleh mahasiswa dan pengelola, serta dapat dikembangkan.

4.2 Model Data yang Sesuai

Model relasional sebagai inti, dilengkapi kolom JSON untuk atribut yang bervariasi (hibrida).

- Data inti yang saling berhubungan (tanaman, kategori_tanaman, perawatan, laporan_kondisi, pengguna) disimpan dalam tabel dengan primary key dan foreign key. Contohnya, kategori_tanaman ke tanaman bersifat satu-ke-banyak, dan tanaman ke perawatan juga satu-ke-banyak. [Sesuaikan relasi dengan ERD kelompok]
- Atribut yang berbeda tiap jenis tanaman (tinggi pohon, warna bunga, khasiat) disimpan di kolom JSON agar tidak ada banyak kolom kosong.

Alasannya, data inti bersifat terstruktur dan membutuhkan integritas, sedangkan sebagian atribut bervariasi sehingga cocok disimpan secara semi-terstruktur.

4.3 Fitur DBMS yang Dibutuhkan

Kebutuhan	Fitur DBMS	Penerapan
Data konsisten dan tidak yatim	Integritas referensial (foreign key)	tanaman_objek.id_kategori ke kategori_tanaman; terbukti saat kategori tidak valid ditolak dengan error #1452

Kebutuhan	Fitur DBMS	Penerapan
Atribut tanaman beragam	Tipe data JSON dan fungsi JSON	Kolom spesifikasi
Operasi data andal	Transaksi ACID (InnoDB)	Perubahan data tidak tersimpan setengah jalan
Pencarian cepat	Indeks	Indeks pada foreign key dan kolom pencarian
Logika di sisi basis data	Stored function/procedure	Misalnya fn_jumlah_laporan()
Keamanan	Manajemen pengguna dan hak akses	Pembatasan akses admin dan pengguna biasa
Keberlanjutan data	Backup dan restore	File backup_modul10.sql dari Modul 10

4.4 Alasan Pemilihan Teknologi

DBMS: MySQL (mesin InnoDB), dikelola dengan phpMyAdmin.

1. Data utama terstruktur dan berelasi, sehingga DBMS relasional paling sesuai.
2. Menyediakan tipe JSON native untuk atribut yang bervariasi.
3. Open source dan gratis, cocok untuk lingkup kampus.
4. Mudah dipelajari dan dikelola (SQL dan phpMyAdmin), serta didukung banyak bahasa pemrograman.
5. Sudah dipakai dan teruji pada praktik kelompok (struktur tabel, CRUD, JSON, backup dan restore).

Pembanding: PostgreSQL unggul jika nantinya dibutuhkan analisis peta yang kompleks, dan MongoDB cocok jika seluruh data berubah-ubah strukturnya. Untuk kebutuhan saat ini keduanya belum diperlukan.

4.5 Dokumentasi dan Repository Git

Tautan repository kelompok: [isi tautan repository]

Struktur repository yang disarankan:

```
green-map-pnm/
├── README.md              (deskripsi proyek, anggota, cara
menjalankan)
├── sql/
│   ├── schema.sql         (CREATE TABLE)
│   ├── data.sql           (INSERT data awal)
│   ├── json_demo.sql      (praktik JSON: INSERT, SELECT, UPDATE,
DELETE)
│   └── backup_modul10.sql
├── docs/
│   ├── diagram_pemodelan_objek_green_map_pnm.png
│   ├── erd.png
│   └── laporan.pdf
└── screenshots/           (bukti hasil praktik)
```

Perintah dasar untuk mengunggah ke repository:

```
git init
git add .
git commit -m "Dokumentasi dan script SQL Green Map PNM"
git branch -M main
git remote add origin https://github.com/<username>/green-map-pnm.git
git push -u origin main
```